

Thirty years of physics, *without* the rewrite.

Legacy C++ and FORTRAN stays exactly where it is. The bindings, the package, the service and the container are generated, so the cost of reaching that code from Python stops being a specialist project and becomes a build step. **\$2,300 to \$19,600 saved per exposed API**, and an open-API partner catalogue that no longer waits on the two people who can compile the FORTRAN.

CONTENTS

01	The problem Three expensive ways to reach the numbers	02	Principle One thin path, then widen it
03	Cost Market rates, and price per exposed API	04	The case Five P&L effects, plus the revenue side
05	Partner API What the open-API initiative needs	06	First move One routine, measured before and after
07	Gaps What is not built, said plainly	08	Evidence Checkable claims, and rate sources

01 THE PROBLEM

The numbers are trapped in code nobody dares touch.

Every new product needs results from code written between 1988 and 2004: a portal, an optimizer, an ML pipeline, a customer API. Today there are three ways to get them, and all three are expensive.

CURRENT OPTION	WHAT IT ACTUALLY COSTS
Rewrite the physics in Python	Months of engineering <i>plus</i> full numerical re-validation by domain experts
Hand-write bindings	A specialist skill, one-off per API, rots on the next signature change
Shell out to a batch executable, parse text files	Brittle, slow, and invisible to monitoring

native2py is the fourth option: **leave the native code exactly as it is and generate the bridge.**

02 PRINCIPLE

Send one thin path all the way through before widening it.

You do not modernize a thirty-year codebase by deciding up front to modernize it. You take one routine end to end (source, binding, wheel, HTTP, container) and see what breaks. Then you widen the path.

That is why `libraries/petro/` is in the repository: ~4,800 lines of period-accurate code with `IMPLICIT` typing, `COMMON` blocks, `INCLUDE` decks and macro-mangled `extern "C"` bridges. The generator was walked against that rather than against a toy header. Its `FINDINGS.md` records what broke on the first attempt, including a bug the tool's own write-up initially misdiagnosed.

THE PROPERTY THAT MAKES THIS SAFE TO ADOPT

The `sim` service is mostly *refusals*. Forward-declared types and raw pointers cause the generator to decline, with a reported reason per symbol. It fails at generation time with an explanation rather than at 3am with a segfault in a worker.

Bindings are no longer written. That is where the money is.

For a real legacy API, a binding runs to eight steps. Resolve the true signatures, which for F77 means working through IMPLICIT typing and COMMON blocks by hand. Decide the argument intent the source never declared. Write the pybind11 or f2py glue with its array shape and ownership rules. Get it to link, where several failure classes only surface at first *import*. Write the package wrapper. Write tests. Validate the numbers against what the native code produced before. Then **do all of it again** at the next signature change.

Steps one through six consume a bindings specialist. Step seven consumes a domain expert, which is why it costs more. Step eight turns a one-time cost into a permanent tax.

What the market pays for those two people

US MARKET, 2026 – PUBLISHED RANGES		
ROLE	SOURCE	ANNUAL
Bindings specialist HPC / scientific-computing engineer	Comparably — average	\$148,808
	ZipRecruiter — 25th–75th	\$113,403–\$188,314
	6figr	\$83,000–\$126,000
Domain expert reservoir engineer / quant / physicist	ZipRecruiter — 25th–75th	\$106,000–\$146,000
	Glassdoor — 25th–75th	\$158,380–\$271,995
	Senior, 8+ yrs — average	\$171,611
Bought in modernization consulting	Enterprise band, realistic	\$150–\$350/hr
	Independent, 8+ yrs — floor	\$1,500/day

Legacy-Fortran modernization sits at the upper end of the \$150–\$350 band, being the specialization that commands the premium. Loading salary by ~1.35 for employer burden and dividing by ~220 working days gives the rates the model runs on:

LOADED DAY RATE		
ROLE	BASIS	PER DAY
Bindings specialist	\$115k–\$190k salary	\$700–\$1,170
Domain expert	\$120k–\$210k salary	\$735–\$1,290
Either, contracted	\$150–\$350/hr	\$1,200–\$2,800

PER API – HAND-WRITTEN			PER API – GENERATED		
STEP	WHO	DAYS	STEP	WHO	DAYS
Understand the signature	Specialist	0.5–3	Add symbol to <code>native2py.yaml</code>	Any engineer	~0
Bindings + build integration	Specialist	1–5	<code>generate</code> / <code>build</code>	Any engineer	~0
Get it to link and import	Specialist	0.5–3	Review refusals + unsupported types	Any engineer	0–1
Package + tests	Specialist	0.5–2	Record and check <code>golden.json</code>	Domain expert	0.25–1
Numerical validation	Domain expert	1–5	Subtotal		0.25–2
Subtotal		3.5–18			

Priced out

COST PER EXPOSED API		
APPROACH	IN-HOUSE	CONTRACTED
Hand-written	\$2,500–\$22,000	\$4,000–\$43,000
Generated	\$200–\$2,500	\$300–\$4,000
Saving per API	\$2,300–\$19,600	\$3,700–\$39,000

THE PETRO LIBRARY ALONE – 12 EXPOSED ROUTINES

At in-house rates, roughly **\$28,000 – \$235,000** of avoided one-time work. Before the recurring maintenance that goes to zero below, and before the validation cost that would otherwise repeat on every rebuild.

About an order of magnitude per API, and the saving falls on the two scarcest resources in the building: the person who can write bindings, and the person who can certify the numbers.

The recurring cost goes to zero

Bindings, CMake, the Python package, the FastAPI service, tests and Dockerfile are all regenerated on demand. What is *not* regenerated is what a human wrote:

```
native/ , native2py.yaml , and the recorded answers.
```

```
native2py generate petro
```

So a native signature change is no longer a ticket assigned to a specialist. It is a re-run. **Binding maintenance stops being a line item.** Across a portfolio of exposed APIs this removes an ongoing $N \times \text{change-frequency} \times \text{specialist-days}$ charge that compounds every year the codebase stays alive.

Validation is spent once, then verified free

```
RECORDED BASELINE – SERVICES/PVT/GOLDEN.JSON  
35 °API    0.75 gas gravity    180 °F    2500 psia  
Rs = 610.696 scf/stb    Bo = 1.34069  
native2py golden verify pvt → 4 entry point(s) unchanged
```

Values a reservoir engineer can check by hand. The expert's time is spent *once*, at record time; every later rebuild, dependency bump, compiler upgrade or refactor is verified by CI at zero marginal expert cost. Without this, each of those events is either a re-validation exercise or an unmanaged risk nobody prices.

A second cost axis: agents pay in context, not hours

The same argument holds when the engineer is an AI coding agent, but the currency changes. For a person the saving is hours. For an agent it is context, which decides whether the task finishes at all.

BINDING ONE 20-METHOD C++ CLASS — ESTIMATED		
PATH	WHAT ENTERS THE CONTEXT WINDOW	OUTPUT TOKENS
Hand-written	~500 lines emitted: pybind11 TU, CMakeLists, pyproject, package init, FastAPI app, tests — then 3–4 compile and link repair rounds, all resident afterwards	20,000–40,000
Generated	One command and a four-line checklist. The generated files never enter context, because there is no reason to read them	300–500

Three effects matter more than the ratio:

- **The repair loop disappears rather than shrinks.** The generator refuses what it cannot bind, with a reason, instead of emitting code that fails to compile.
- **golden verify is a correctness signal the agent cannot fake.** Left to itself, an agent writes tests from its own reading of the source, validating its interpretation rather than the numbers.
- **Generated output is identical at 80% context.** A hand-written binding degrades as the window fills.

ON THESE TOKEN FIGURES

Estimates derived from the size of the generated artifacts, not a measured benchmark. The ratio is not a close call; the absolute numbers are not data.

Where cost does not go away

- **Writing the native code** — unchanged. Nothing is rewritten.
 - **Choosing what to expose** — a design decision, and it should be. Exposing everything is explicitly a non-goal.
 - **Coarse-graining chatty APIs** — a call-per-datapoint API is slow through any binding layer. That is an API design cost, not a tooling cost.
 - **The platform gaps in §07** — auth, isolation, CI/CD and Kubernetes are real work that is not yet done.
-

What this buys, in the order it hits the P&L.

I Migration cost collapses from a rewrite to a config file

The unit of work becomes “expose these twelve routines,” not “port the PVT module and prove it still works.” No physics is re-derived, so no physicist has to re-derive it. That is the difference between a multi-quarter programme and a sprint.

II Numerical risk is bounded

“Did the re-host change our answers?” goes from expert-weeks to a green check in CI. For anything regulated, contractual, or reported to a customer, this is usually the line that gets the project approved at all, and the strongest defence if the answer is ever challenged afterwards.

III Key-person risk drops off the critical path

Today the two people who can compile the FORTRAN gate every downstream project, and one is closer to retirement than to the roadmap. Afterwards, Python developers write `from petro import ...` and never learn f2py. The scarce skill is no longer required for each new feature, which is a throughput gain across every team consuming these numbers.

IV It is incremental, therefore cancellable

Phase one is embedding: bindings inside the existing application, no deployment change, no architecture review. Containerizing and extracting to services are separate later decisions, and the native implementation does not change across them. You can fund one service, stop, and still hold something of value. A rewrite has no useful intermediate state.

V Optionality on transport, bought cheaply

The intermediate representation sits between the language parsers and the binding generators so that `Python → pybind11 → C++` can later become `Python → gRPC → C++ service` without changing calling code. That mode is not built. The seam is, and the seam is the expensive part to retrofit.

The revenue side, usually the larger number

- **Time-to-market** on every product that depends on these numbers. Anything currently queued behind a binding specialist ships sooner.
- **Products that are never proposed** because “we'd have to touch the FORTRAN.” Lowering that cost changes what reaches the roadmap.
- **Monetizing existing computation.** A validated correlation library reachable over HTTP is a sellable or licensable asset. A batch executable on a build server is not.
- **Extending the life of a paid-for asset.** The alternative to re-hosting is eventual forced replacement — a large future capital cost, deferred or removed.

Sizing it for this organization — three numbers, obtainable in a day

INPUT	QUESTION	WHERE TO GET IT
N	How many native APIs do downstream teams want and not have?	Ask the teams, not the platform group
D	How long does a new binding take, wall-clock, from ask to usable?	The last three tickets
C	Annual spend maintaining existing bindings and re-validating after changes?	Engineering budget

NEAR-TERM SAVING

$$N \times (\text{hand-written days} - \text{generated days}) \times \text{loaded rate}$$
, plus C recurring, plus the value of reducing D . That last one is usually what the business actually cares about, because it is the one blocking revenue.

The shortest path to the open-API partner initiative.

TO BE MADE SPECIFIC BEFORE THIS IS CIRCULATED

This section is written against the general shape of a partner-API programme. It needs five facts to name real bottlenecks instead of describing a category: the partners or tier being targeted; which computations they want to call; the endpoint count and any committed date; the commercial model; and who owns auth, rate limiting and SLAs today.

The partner programme needs computational endpoints that partners can call, version against, and rely on contractually. Those computations exist today only as FORTRAN routines and batch executables, so every partner endpoint is gated on a bindings specialist and a validation cycle. That is the bottleneck this removes.

REQUIREMENT → WHAT IS ALREADY PRODUCED	
THE PARTNER API NEEDS	WHERE IT COMES FROM
A machine-readable contract partners generate clients from	FastAPI emits OpenAPI schema automatically for every exposed service. No separate spec to write, and none to drift
A surface that does not break when native code changes	Python is the stable interface; the implementation is not. Internals evolve behind <code>from petro import ...</code> , and a breaking native change fails <i>binding generation</i> rather than reaching a partner
Numbers you can stand behind contractually	<code>golden.json</code> pins the returned values. A disputed result meets a committed, hand-verifiable baseline and a CI check proving nothing moved
Independent versioning and release per endpoint	Every service is its own wheel, version and container. One partner endpoint ships without rebuilding the others
Endpoints added at partner-request speed	Adding a routine to the catalogue is a config entry and a regenerate

EFFECT ON THE INITIATIVE'S TIMELINE

Each new partner endpoint currently carries **3.5–18 specialist and expert days** of binding and validation before anyone writes a line of API surface. Generated, that is **0.25–2 days**. Across a catalogue of a dozen endpoints, that is the difference between a multi-quarter build-out and one sprint of configuration, and the removed work is the work only two people in the building can do.

The second-order effect matters as much. The partner roadmap stops being shaped by *what we can afford to bind* and starts being shaped by *what partners will pay for*. Endpoints currently unproposed because the binding cost cannot be justified become viable.

CAVEAT, AND HERE IT IS ON THE CRITICAL PATH

A partner-facing API is the case the gaps in §07 do not yet cover: no auth, no rate limiting, no exception handling, and a native segfault takes the worker down. Tolerable for internal phase-one embedding; blocking for an external partner endpoint.

So the sequencing is two workstreams in parallel, not one after the other:

① Use native2py now to remove the

binding bottleneck and build the internal catalogue. That value is available today.
and
validation

② Fund the platform layer — auth, rate limiting, segfault isolation, deployment templates — as a named workstream

for , since the initiative is what makes it mandatory.
the
partner
initiative

What is no longer on the critical path is the thirty-year-old FORTRAN, which was always the part nobody could schedule.

Propose a path, not a programme.

- 1 Pick
one routine that two or three downstream teams are currently waiting on.
- 2 Take it end to end: expose, build, record golden values, ship the wheel to one of those teams.
- 3 Measure the thing that moves budget — how long that team waited

before, versus

after .

- 4 Decide about the remaining routines with that number in hand.

Total exposure if it fails: one routine, a few days, and a findings entry that makes the next attempt cheaper.

What must be said before anyone approves this.

Approving on an incomplete picture is how a project loses credibility in month three. As of today:

KNOWN GAPS — NOT YET BUILT

- Generated services have **no auth, no rate limiting, and no exception handling**.
- **Native code that segfaults takes the worker down with it.** There is no process isolation yet.
- `infrastructure/docker/` and `infrastructure/kubernetes/` are **empty** — no CI/CD pipelines or Kubernetes manifests are generated.

Production-ready today as a build-time tool and as a phase-one embedding story. Not yet production-ready as a public-facing service platform.

The gaps are ordinary platform engineering — auth, a segfault-isolation strategy, deployment templates. They are scoped in `production-readiness.md`, which grades the tool against the specification's own requirements rather than against a sales pitch. They are simply not done, and they should be costed as part of any decision to go beyond phase one.

For anyone who would rather check than take it on trust.

- **Eight generated services** across C++ and Fortran, each an independently buildable wheel and container — including `sim`, which is mostly documented refusals, and `petro`, which exposes twelve routines from seven F77 decks plus an F90 facade.
- **220 tests** in the generator's own suite, plus a second full run against the fallback C++ parser backend.
- **Real end-to-end builds**, rather than unit tests alone: generated CMake fed to real `cmake`, `clang++` and `gfortran`, the extension imported, the FastAPI service exercised over HTTP. Several real bugs were found this way and not by inspection — f2py's module nesting, a missing-`.cpp` link failure that only appears at first import, and a golden harness that replayed stateful F77 routines out of order.
- **A recorded numerical baseline** a domain expert can verify by hand.

Sources for the rate figures in §03

Public US market data, retrieved 22 August 2026. Ranges vary by methodology; the model uses the mid-band deliberately rather than the most favourable figure.

- **Comparably – HPC Software Engineer** (<https://www.comparably.com/salaries/salaries-for-hpc-software-engineer>)
- **ZipRecruiter – HPC Engineer** (<https://www.ziprecruiter.com/Salaries/Hpc-Engineer-Salary>)
- **6figr – HPC Engineer** (<https://6figr.com/us/salary/high-performance-computing-engineer--t>)
- **ZipRecruiter – Reservoir Engineer** (<https://www.ziprecruiter.com/Salaries/Reservoir-Engineer-Salary>)
- **Glassdoor – Reservoir Engineer** (https://www.glassdoor.com/Salaries/reservoir-engineer-salary-SRCH_K00,18.htm)
- **PayScale – Reservoir Engineer** (https://payscale.com/research/US/Job=Reservoir_Engineer/Salary)
- **SalaryExpert – Reservoir Engineer** (<https://www.salaryexpert.com/salary/job/reservoir-engineer/united-states>)
- **Application Modernization Consulting Rates – 2026 benchmark** (<https://softwaremodernizationservices.com/insights/application-modernization-consulting-rates/>)
- **Cleveroad – Software consulting rates 2026** (<https://www.cleveroad.com/blog/software-consulting-rates/>)

native2py – business case Companions: design.md · README.md · production-readiness.md
Source: github.com/ravikings/ThinkDeck · [haliburtion/](#) Rev. 2026-08-22